LAB 4

First Backend: FastAPI, Pydantic, and Depends

Goal

Create a standalone backend project, course-catalog-api, on FastAPI and move into it the course catalog that has lived in lib/courses.ts until now: describe a course with a Pydantic model, implement the “course list” and “single course” routes, and add filtering, sorting, and pagination. We're not touching the frontend today — you'll verify the backend purely through the automatic documentation at /docs. We'll connect it to Next.js in Lab 5.

Before you start

- Python 3.10 or newer — check with python --version (on macOS and Linux often python3 --version): FastAPI requires 3.10+, and the str | None syntax from the lecture only works from that version too;
- git and a GitHub account — the same ones as in Lab 1;
- a code editor (VS Code with the Python extension is recommended);
- Lecture 4's handout open nearby — today you'll repeat its sections 2–6 almost word for word;
- lib/courses.ts from Lab 1 — this is exactly the course catalog you're rewriting in Python today.

If you're opening a terminal for Python for the first time

The text below uses the commands python, pip, and venv. Here's the minimum you need today:

- pip is Python's package installer, the counterpart of npm in the JavaScript world;
- venv is a “virtual environment”: a .venv folder inside the project where pip installs packages for this project only, not for the whole computer (conceptually, the counterpart of node_modules);
- the environment must be activated in every new terminal window; the sign of success is a (.venv) prefix at the start of the prompt;
- on Windows the command is usually called python, on macOS and Linux — python3; once the environment is activated, plain python works everywhere;
- if a command fails with an error — read the last line of the error; most of the time it says outright what's wrong.

What you'll build

A backend with three routes and documentation that FastAPI builds on its own:

```
route map
GET /                    — checks that the server is alive
GET /courses             — course list: filtering, sorting, pagination
GET /courses/{course_id} — a single course or a 404 error
GET /docs                — interactive docs (generated automatically)
```

The project structure by the end of the lab:

```
project structure
course-catalog-api/
├── .venv/            # virtual environment — never goes into git
├── .gitignore
├── README.md
├── requirements.txt
├── models.py         # the Pydantic Course model
├── data.py           # data: the same six courses as in lib/courses.ts
└── main.py           # the app and its routes
```

1. Environment and Installation

Create a project folder, go into it, and create a virtual environment. The commands depend on your system:

```
terminal — macOS / Linux
mkdir course-catalog-api
cd course-catalog-api
python3 -m venv .venv
source .venv/bin/activate
```

```
terminal — Windows (PowerShell)
mkdir course-catalog-api
cd course-catalog-api
python -m venv .venv
.venv\Scripts\Activate.ps1
```

> **If something goes wrong:** PowerShell may reply that script execution is disabled — run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once and repeat the activation (or open the regular command prompt, cmd, and activate the environment with .venv\Scripts\activate). On Ubuntu and Debian, creating the environment sometimes requires installing a package once: sudo apt install python3-venv.

A (.venv) prefix should now appear at the start of your terminal line. Now install FastAPI:

```
terminal
pip install "fastapi[standard]"
```

> **Quotes are required:** in zsh (the default macOS terminal), square brackets without quotes are treated as a filename pattern, and the command won't work. The [standard] extra pulls in everything you need at once — including the Uvicorn server and the fastapi command.

Record the dependency in a file — that way any classmate can repeat your installation with one command — and set up git right away, so the environment folder never ends up in the repository:

```
requirements.txt
fastapi[standard]
```

```
.gitignore
.venv/
__pycache__/
```

```
terminal
git init
git add .
git commit -m "chore: bootstrap FastAPI project"
```

Check yourself: pip show fastapi prints lines starting with Name: fastapi and Version: …, and git status doesn't mention the .venv folder. If it does, there's a mistake in .gitignore (check the leading dot and the trailing slash).

2. First Run

Create main.py and declare a minimal app — exactly what we wrote in section 2 of the lecture:

```
main.py
from fastapi import FastAPI

app = FastAPI(title="Course Catalog API")


@app.get("/")
def read_root():
    return {"message": "Course Catalog API is running"}
```

Start the dev server:

```
terminal
fastapi dev main.py
```

The terminal will show the server's address (usually http://127.0.0.1:8000) and a link to the documentation. Open both addresses in your browser: at / you'll see JSON, and at /docs — an interactive Swagger page. Expand the GET / route, click Try it out, then Execute — the response arrives right on the page, with no Postman and no terminal.

Don't stop the server: leave fastapi dev running in this terminal window, and open a second window for other commands (don't forget to activate .venv there too). The server restarts itself every time you save a file.

Check yourself: change the message text in read_root, save the file, and refresh the page in your browser — the new text should appear without manually restarting the server.

3. The Course Model and Data

Remember type Course from Lecture 1? Today you describe the same object shape in Python. Create models.py: the Course class inherits from BaseModel (section 4 of the lecture) and contains the fields from the table:

Field	Type	Default	TypeScript counterpart
id	str	— (required)	id: string

Field	Type	Default	TypeScript counterpart
title	str	— (required)	title: string
description	str	— (required)	description: string
credits	int	— (required)	credits: number
is_elective	bool	False	isElective: boolean
likes	int	0	likes: number

```
models.py — skeleton
from pydantic import BaseModel


class Course(BaseModel):
    id: str
    # TODO: the remaining fields from the table above
```

> **Names in JSON:** Python convention is snake_case, so the field is called is_elective, while in the TypeScript type from Lecture 1 it's isElective. The JSON your backend returns will have is_elective. We'll sort this difference out neatly in Lab 5 — for today, just remember that it exists.

A two-minute experiment: validation at runtime

Once the model is ready, start the Python interpreter with the python command (the environment must be active) and type two lines — an attempt to create a course with credits="five":

```
python (interactive mode)
from models import Course
Course(id="x", title="t", description="d", credits="five")
```

Pydantic will refuse to create the object and explain why: the end of the error contains the line credits and the phrase Input should be a valid integer. Now replace "five" with "5" — the object gets created: by default, Pydantic neatly converts the string "5" to the number 5. This is exactly the validation at runtime that the lecture talked about. Exit the interpreter with exit().

Data

Now the data. This is a tool, not the point of the assignment: create data.py and copy it — the same six courses that were in lib/courses.ts. The file has two parts; first, the data:

```
data.py — part 1: data
from models import Course

COURSES: list[Course] = [
    Course(
        id="modern-frontend",
        title="Modern Frontend: React & Next.js",
        description="React 19, Server Components, and the App Router.",
        credits=5,
        # is_elective is not set — the default value False applies
        likes=24,
    ),
    Course(
        id="backend-fastapi",
        title="Backend Foundations: FastAPI",
        description="Async REST APIs in Python with FastAPI and Pydantic.",
        credits=5,
        likes=19,
    ),
    Course(
        id="databases-postgresql",
        title="Relational Databases: PostgreSQL",
        description="Schemas, SQLAlchemy, and migrations with Alembic.",
        credits=5,
        likes=15,
    ),
    Course(
        id="api-design",
        title="API Design: REST vs GraphQL",
        description="Comparing REST and GraphQL in one hands-on project.",
        credits=4,
        is_elective=True,
        likes=11,
    ),
    Course(
        id="web-security",
        title="Web Security Essentials",
        description="JWT/OAuth2, XSS, CSRF, and SQL injection defense.",
        credits=4,
        likes=21,
    ),
    Course(
        id="ai-integration",
        title="AI/LLM Integration",
        description="LLM features in an app via the OpenAI API.",
        credits=5,
        is_elective=True,
        likes=32,
    ),
]
```

Note that for four courses the is_elective field isn't set at all — the default value False from your model kicked in. Now, at the end of the same file, after two blank lines, add the two helper functions mentioned in the lecture: get_all_courses returns a copy of the list, and find_course looks a course up by id and returns None if there isn't one.

```
data.py — part 2: functions (at the end of the file)
def get_all_courses() -> list[Course]:
    return list(COURSES)


def find_course(course_id: str) -> Course | None:
    for course in COURSES:
        if course.id == course_id:
            return course
    return None
```

```
terminal
python -c "import data; print(len(data.COURSES))"
```

> **Check yourself:** the command above prints 6. If you see a ValidationError instead of a number, Pydantic found a mismatch between your model and the data (for example, a typo in a field name or a type that didn't match); read the error — it names the problem field.

4. The Course List: GET /courses

Now the first real route. Extend main.py: imports of the data and the model, and the GET /courses route with response_model — as in section 4 of the lecture:

```
main.py — skeleton
from fastapi import FastAPI

from data import find_course, get_all_courses
from models import Course

app = FastAPI(title="Course Catalog API")


@app.get("/")
def read_root():
    return {"message": "Course Catalog API is running"}


@app.get("/courses", response_model=list[Course])
def list_courses():
    return []  # TODO: return all courses (hint: get_all_courses)
```

find_course is imported in advance — you'll need it in the next section. Save the file and open /courses in your browser: you should get a JSON array of six objects. Then go to /docs, expand GET /courses and look at Responses → Example Value: the documentation already knows the shape of the response. Scroll down to the Schemas section — there's your Course model, retold by FastAPI.

> **Check yourself:** each of the six objects has the keys id, title, description, credits, is_elective, and likes; the order is still the same as in data.py — modern-frontend comes first.

5. A Single Course and the 404 Error

Add a route for a single course. A path parameter is declared in curly braces — the syntax is familiar from Next.js dynamic segments. If there's no course with that id, you need to return a 404 error, not an empty response. In FastAPI this is done with the HTTPException exception — the counterpart of notFound() from Lab 1:

```
main.py — route skeleton
from fastapi import FastAPI, HTTPException   # add HTTPException

@app.get("/courses/{course_id}", response_model=Course)
def get_course(course_id: str):
    course = find_course(course_id)
    # TODO: if the course isn't found (course is None), raise
    #   raise HTTPException(status_code=404, detail="Course not found")
    return course
```

New: HTTPException: this is the only thing that wasn't in the lecture. The expression raise HTTPException(status code=404, detail="...") interrupts the function and immediately sends the client a response with a 404 code and a body like {"detail": "..."}. Code 404 means “not found” — the same meaning as notFound() in Next.js.

A trap from the lecture — see it with your own eyes

Temporarily change course_id: str to course_id: int, save, and open /courses/web-security. Instead of the course you'll get a 422 error with the message Input should be a valid integer: FastAPI didn't let the request into your function, because course ids are strings. Change it back to str. This is how you confirm that a type annotation is a real check, not a hint for your editor.

Check yourself: /courses/web-security returns one course; /courses/nope returns a 404 code and the body {"detail": "Course not found"}.

6. Query Parameters: Filtering and Sorting

Extend GET /courses with two optional parameters (section 5 of the lecture). is_elective — show only elective (true) or only required (false) courses. sort — the order of results: popular (by likes, descending; this is the default) or title (alphabetically by title).

```
main.py — skeleton
@app.get("/courses", response_model=list[Course])
def list_courses(is_elective: bool | None = None, sort: str = "popular"):
    courses = get_all_courses()
    # TODO 1: if is_elective is provided, keep only the matching courses
    # TODO 2: if sort == "title", sort by c.title,
    #         otherwise by c.likes, highest first
    return courses
```

- a filter is conveniently written as a list comprehension: [c for c in courses if ...];
- sorting: sorted(courses, key=lambda c: c.title); for reverse order, add reverse=True;
- think: how does the check if is_elective: differ from if is_elective is not None:, and what happens on a request with ?is_elective=false?

Verify the result against the table. Note that the default order has changed — it's now sort=popular, and that's expected.

Request	Expected result
/courses	6 courses; ai-integration first (32 likes), api-design last (11)
/courses?is_elective=true	2 courses: ai-integration, api-design
/courses?is_elective=false	4 courses: modern-frontend, web-security, backend-fastapi, databases-postgresql
/courses?sort=title	6 courses alphabetically by title: ai-integration, api-design, backend-fastapi, modern-frontend, databases-postgresql, web-security

7. Pagination with Depends

The last piece is a dependency, as in section 6 of the lecture. Create a pagination function with the parameters page (default 1) and page_size (default 20) that returns a dictionary with the keys offset and limit, and plug it into GET /courses via Depends. The route shouldn't calculate the offset itself — it asks for a ready-made result.

```
main.py — skeleton
from fastapi import Depends, FastAPI, HTTPException   # add Depends


def pagination(page: int = 1, page_size: int = 20):
    # TODO: return {"offset": ..., "limit": ...}
    # offset is how many items to skip: (page - 1) * page_size
    return {}


@app.get("/courses", response_model=list[Course])
def list_courses(
    is_elective: bool | None = None,
    sort: str = "popular",
    p: dict = Depends(pagination),
):
    courses = get_all_courses()
    # filter and sorting from section 6 — leave them as they are
    # TODO: return only the needed “slice” of the list:
    #   courses[offset : offset + limit]
    return courses
```

The order of operations matters: filter first, then sort, and only then slice out the page — otherwise you'll be slicing something other than what you filtered.

> **Modern Depends syntax:** in the latest FastAPI documentation you'll see p: Annotated[dict, Depends(pagination)] instead of p: dict = Depends(pagination). They're the same thing: Annotated has been recommended since version 0.95.0, and the form from the lecture still works. If you'd like to try the new one, import Annotated from the typing module and make p the first parameter (without a default value it can't come after parameters that have one — Python raises a SyntaxError).

Request	Expected result
/courses?page=1&page_size=2	ai-integration, modern-frontend
/courses?page=2&page_size=2	web-security, backend-fastapi

Request	Expected result
/courses?page=3&page_size=2	databases-postgresql, api-design
/courses?page=4&page_size=2	an empty list []
/courses	all 6 courses (page_size defaults to 20)

8. Verifying via /docs and the README

You didn't write a test client — the documentation serves as one. Go through the steps:

1. open /docs and make sure there are exactly three routes: GET /, GET /courses, GET /courses/{course_id};
2. expand GET /courses and click Try it out: fill in is_elective (a true/false dropdown), sort, page, page_size and click Execute; study the Curl, Request URL, and Response body blocks — the Request URL is exactly the address Next.js will call later;
3. expand GET /courses/{course_id}, enter nope, click Execute, and confirm the response code is 404;
4. scroll /docs down to Schemas and find Course — that's your model from models.py;
5. open /redoc and /openapi.json — the latter is the machine-readable “passport” of the API that both documentation pages are built from.

Create a README.md in three parts: what the project is (one line); how to run it (create the environment, pip install -r requirements.txt, fastapi dev main.py); and a “What was verified” section — a list of requests from the tables above with what each of them returned.

Acceptance Checklist

Before you submit, run through this list — it's exactly what will be checked:

- ☐ fastapi dev main.py starts without errors and /docs opens;
- ☐ /docs shows exactly three routes: GET /, GET /courses, GET /courses/{course_id};
- ☐ Course is described via BaseModel with the fields from the table (is_elective defaults to False, likes defaults to 0);
- ☐ GET /courses returns 6 courses, with ai-integration first by default;
- ☐ /courses/web-security returns a course, and /courses/nope returns a 404 error with detail;
- ☐ /courses?is_elective=true gives exactly 2 courses, and /courses?is_elective=false gives exactly 4;
- ☐ /courses?sort=title returns the courses alphabetically by title;
- ☐ /courses?page=2&page_size=2 gives web-security and backend-fastapi;
- ☐ pagination is wired in through Depends(pagination), not calculated directly inside the route;
- ☐ the repository has no .venv folder, has requirements.txt and README.md, and has at least 4 meaningful commits.

Grading Rubric

Criterion	Points
Environment: venv, fastapi[standard] installed, server starts, .venv not in git	1
The Pydantic Course model and the data from data.py	2
GET /courses with response_model=list[Course]	1
GET /courses/{course_id}: course found — a response, not found — 404 via HTTPException	2
Query parameters is_elective and sort work per the table in section 6	1
Pagination via the Depends(pagination) dependency works per the table in section 7	1
README describing how to run it, with a list of verified requests	1
GitHub repository with a history of meaningful commits	1

Total: 10 points. Adjust the scale to your own grading system as needed.

Common Mistakes

fastapi: command not found: almost always means the environment isn't activated in this terminal window. Check for the (.venv) prefix at the start of the line; if it's missing, repeat the activation command from section 1.

ModuleNotFoundError: No module named 'models': commands must be run from the project folder, where main.py, models.py, and data.py sit side by side. cd into it and run again.

course_id: int instead of str: course ids are strings like web-security. With the int type, any real id gives a 422 error: a type in a signature is a real check, not a hint.

if is_elective: instead of is not None: the value false is “falsy” in Python, so such a filter silently ignores ?is_elective=false and returns all 6 courses instead of 4.

.venv folder in git: if git status shows .venv/ — your .gitignore is wrong. Already committed it? Run git rm -r --cached .venv, fix .gitignore, and make a new commit.

Address already in use: port 8000 is taken: another fastapi dev is already running (or some other program is using it). Stop the previous server with Ctrl+C, or start a new one on a different port: fastapi dev main.py --port 8001.

Bonus Tasks (optional)

If the core part is done early — pick any, no limit on how many:

1. replace sort: str with sort: Literal["popular", "title"] (Literal is imported from typing): /docs will show a dropdown, and a ?sort=banana request will return a 422 error instead of quietly sorting “by popularity”;
2. protect pagination with Query: page with the constraint ge=1, page_size with ge=1 and le=100 (Query is imported from fastapi, most conveniently through Annotated). Requests with page=0 and page_size=1000 should give 422;

3. add search by title — a q parameter, a case-insensitive substring: ?q=react gives modern-frontend, and ?q=api gives backend-fastapi and api-design (hint: q.lower() in c.title.lower()). This is exactly the kind of search that will be useful for the client-side search from Lecture 2;
4. add a GET /stats route with your own Pydantic model Stats (fields total, total_credits, electives) — this is the very /api/stats from the CourseStats example in Lecture 2. Expected response: total — 6, total_credits — 28, electives — 2.

How to Submit

Create a course-catalog-api repository on GitHub (public, or with access for your instructor) and push the project into it.

Commit as you go, not in one chunk at the end. Naming guideline: chore: bootstrap FastAPI project → feat: hello-world endpoint → feat: Course model and data → feat: GET /courses → feat: GET /courses/{id} with 404 → feat: filter and sort → feat: pagination via Depends → docs: README. The required minimum is 4 commits.

Send the repository link however your instructor specifies (LMS / course chat).

Deadline: before Lecture 5 begins, unless your instructor states otherwise.